TACOMANDA — FLUJOS CQRS Y TARJETAS CRC

Anexo de la fase de diseño · Versión en blanco y negro · Tipografía Times New Roman

Este documento reorganiza en formato académico el contenido del PDF inicial de tarjetas CRC. Se actualizaron las descripciones de envío de noControl para alinearlas con el contrato Swagger: la identidad del empleado debe obtenerse del JWT. El proyecto continúa en fase de diseño; los componentes descritos aún no acreditan una API funcional.

CQRS separa comandos (operaciones que alteran el estado) y consultas (operaciones que solo leen). Las tarjetas CRC describen clases, responsabilidades y colaboradores previstos.

FLUJO 1. INICIO DE SESIÓN

Comando: IniciarSesion. Consulta prevista: ObtenerPermisosDeEmpleado.

Pasos del proceso

1. El empleado captura su número de control y contraseña en la pantalla de inicio de sesión.
2. El cliente Angular envía POST /api/auth/login con las credenciales.
3. El controlador recibe LoginRequest y crea el comando para la capa Application.
4. LoginValidator revisa la forma de los datos: campos no vacíos y número de control válido.
5. LoginHandler recupera al empleado mediante IEmpleadoRepository.
6. El servicio IPasswordHasher verifica la contraseña contra el hash BCrypt almacenado.
7. Si se autentica, ITokenService genera el JWT con la identidad y el rol del empleado.
8. El handler devuelve el resultado y la API lo transforma en LoginResponse o en un error HTTP.
9. El frontend almacena la sesión y obtiene los permisos incluidos en LoginResponse para navegar al módulo del rol.
10. El contrato actual no define un endpoint independiente de consulta de permisos: esta consulta aparece como posibilidad en el diagrama CRC inicial.

Tarjetas CRC

Clase	Responsabilidad	Colaboradores
PantallaDeLogin	Capturar las credenciales y mostrar el resultado al empleado.	ClienteDeAutenticacion, AlmacenDeSesionDelNavegador, EnrutadorDePantallas
ClienteDeAutenticacion	Viajar entre la pantalla y el servidor con los comandos y consultas de acceso.	ControladorDeInicioDeSesion
ControladorDeInicioDeSesion	Traducir la petición HTTP a un comando o consulta, y el resultado a una respuesta HTTP.	ValidadorDeComandoIniciar Sesion, DespachadorDeComandos, DespachadorDeConsultas
ComandoIniciarSesion	Representar la intención de entrar (número de control y contraseña), sin lógica.	Ninguno
ValidadorDeComandoIniciarSesio n	Garantizar que el comando tenga la forma correcta antes de procesarlo.	Ninguno

Clase	Responsabilidad	Colaboradores
DespachadorDeComandos	Entregar cada comando a su manejador correspondiente.	ManejadorDeComandoIniciar Sesion (y los demás manejadores de comandos)
DespachadorDeConsultas	Entregar cada consulta a su manejador correspondiente.	ManejadorDeConsultaPermisos (y los demás manejadores de consultas)
ManejadorDeComandoIniciarSesion	Orquestar los pasos para autenticar a un empleado y abrir su sesión.	RepositorioDeEmpleados, Empleado, GeneradorDeSesion, EscritorDeBitacoraDeAccesos
RepositorioDeEmpleados	Recuperar empleados completos desde la base de datos para los comandos.	Empleado (Administrador, Cajero, Mesero)
Empleado (Administrador, Cajero, Mesero)	Conocer sus datos y permisos propios, y verificar su contraseña.	VerificadorDeContrasena
VerificadorDeContrasena	Decidir si una contraseña escrita corresponde a la guardada.	Ninguno
GeneradorDeSesion	Crear una sesión válida para un empleado autenticado.	Ninguno
EscritorDeBitacoraDeAccesos	Dejar registro de cada intento de acceso.	Ninguno
ManejadorDeConsultaPermisos	Armar la vista de permisos de un empleado, solo leyendo.	LectorDeEmpleados
LectorDeEmpleados	Entregar vistas de empleados para las consultas, sin modificarlos.	Ninguno
AlmacenDeSesionDelNavegador	Recordar quién está dentro mientras la app esté abierta.	Ninguno
EnrutadorDePantallas	Decidir a qué pantalla va cada rol.	AlmacenDeSesionDelNavegador

Referencia de origen: flujos-cqrs-crc_original.pdf, páginas 1 y 2.

FLUJO 2. ENVÍO DE COMANDA Y COCINA

Comandos: EnviarComanda, AvanzarEstadoDeComanda. Consultas: listar platillos, mesas y comandas.

Pasos del proceso

1. La pantalla del mesero consulta el menú disponible y las mesas mediante rutas GET.
2. El carrito reúne id de platillo, cantidad y nota antes de guardar cambios.
3. Al confirmar, el cliente envía POST /api/comandas con mesaNumero e items. No incluye noControl ni precios.
4. El backend identifica al mesero desde el JWT y valida que existan ítems y cantidades positivas.
5. El handler consulta la mesa y los platillos, y comprueba disponibilidad y estado activo.
6. La lógica de negocio calcula cada subtotal usando precios del servidor.
7. Se crea la comanda en estado abierta y sus detalles.
8. La operación guarda comanda y detalles y cambia la mesa a ocupada como transacción prevista.
9. Si todo se registra, la API devuelve el identificador de la comanda (201); si falla, responde con un error.

10. Cocina consulta las comandas mediante GET /api/comandas y filtra opcionalmente por estado.

11. Cuando cocina cambia el estado, se envía PUT /api/comandas/{id}/estado; solo se permitirán transiciones válidas desde abierta a en_preparacion y lista.

12. El refresco periódico del tablero es una propuesta del prototipo, no una operación HTTP adicional.

Tarjetas CRC

Clase	Responsabilidad	Colaboradores
PantallaDeMesero	Mostrar el menú y las mesas y capturar lo que ordena el cliente.	ClienteDeComandas, ConstructorDeCarrito
ConstructorDeCarrito	Acumular platillos, cantidades y notas antes de enviar.	Ninguno
ClienteDeComandas	Viajar entre las pantallas y el servidor con comandos y consultas de comandas.	ControladorDeComandas
ControladorDeComandas	Traducir peticiones HTTP a comandos o consultas, y los resultados a respuestas.	ValidadorDeComandoEnviarComanda, DespachadorDeComandos, DespachadorDeConsultas
ComandoEnviarComanda	Representar la intención de mandar una orden (mesa, mesero, líneas), sin lógica.	Ninguno
ValidadorDeComandoEnviarComanda	Garantizar la forma correcta de la orden antes de procesarla.	Ninguno
ManejadorDeComandoEnviarComanda	Orquestar la creación de una comanda.	RepositorioDeMesas, RepositorioDePlatillos, CalculadoraDeSubtotales, Comanda, RepositorioDeComandas
CalculadoraDeSubtotales	Obtener el subtotal de cada línea con el precio vigente del servidor.	Ninguno
Comanda	Conocer sus detalles y decidir qué cambios de estado son válidos.	DetalleDeComanda
DetalleDeComanda	Representar una línea de la comanda (platillo, cantidad, nota, subtotal).	Ninguno
RepositorioDeComandas	Guardar y recuperar comandas completas como una unidad.	Comanda
RepositorioDeMesas	Recuperar mesas y guardar su estado.	Ninguno
RepositorioDePlatillos	Recuperar platillos con su precio vigente para los comandos.	Ninguno
ManejadorDeComandoAvanzarEstadoDeComanda	Aplicar el cambio de estado pedido por cocina.	RepositorioDeComandas, Comanda
PantallaDeCocina	Mostrar el tablero y capturar el avance de cada comanda.	ClienteDeComandas
ManejadorDeConsultaMenu	Armar la vista del menú disponible, solo leyendo.	LectorDeMenu
ManejadorDeConsultaMesas	Armar la vista de mesas disponibles, solo leyendo.	LectorDeMesas
ManejadorDeConsultaTableroDeCocina	Armar el tablero de comandas activas para cocina.	LectorDeTableroDeCocina

Clase	Responsabilidad	Colaboradores
LectorDeMenu	Entregar los platillos activos en forma de vista.	Ninguno
LectorDeMesas	Entregar las mesas y su estado en forma de vista.	Ninguno
LectorDeTableroDeCocina	Entregar las comandas activas con sus tiempos de espera.	Ninguno

Referencia de origen: flujos-cqrs-crc_original.pdf, páginas 3 y 4.

FLUJO 3. COBRAR UNA COMANDA

Comando: CobrarComanda. Consultas: ObtenerComandasPorCobrar y ObtenerTicketDeCobro.

Pasos del proceso

1. La pantalla de caja consulta las comandas y presenta aquellas listas para cobrar.
2. El cajero selecciona una comanda y captura el tipo de pago, tipo de pedido y monto recibido.
3. Se envía POST /api/comandas/{id}/cobrar con CobroRequest. El JWT identifica al cajero; no se envía noControl ni total.
4. El controlador valida la estructura y delega el comando al handler.
5. La capa Domain verifica que la comanda esté lista y no haya sido cerrada, cancelada o cobrada.
6. El sistema calcula el total usando subtotales almacenados por el servidor.
7. Se aplica la política de pago: efectivo exige monto suficiente y calcula cambio; tarjeta se considera importe exacto según el diseño.
8. Se registra el pedido, se cierra la comanda y se libera la mesa en la misma transacción prevista.
9. La respuesta del comando devuelve el idPedido (201), no el total ni el cambio.
10. La pantalla solicita GET /api/pedidos/{id}/ticket para consultar la información completa del comprobante.
11. El frontend podrá presentar o imprimir el ticket; esa funcionalidad no constituye un endpoint nuevo.

Tarjetas CRC

Clase	Responsabilidad	Colaboradores
PantallaDeCaja	Mostrar las comandas por cobrar y capturar el pago.	ClienteDeCaja, ImpresorDeTicket
ClienteDeCaja	Viajar entre la pantalla y el servidor con comandos y consultas de cobro.	ControladorDeCobros
ControladorDeCobros	Traducir peticiones HTTP de caja a comandos o consultas.	ValidadorDeComandoCobrar Comanda, DespachadorDeComandos, DespachadorDeConsultas
ComandoCobrarComanda	Representar la intención de cobrar una orden con un pago (sin total), sin lógica.	Ninguno

Clase	Responsabilidad	Colaboradores
ValidadorDeComandoCobrarComanda	Garantizar la forma correcta del cobro antes de procesarlo.	Ninguno
ManejadorDeComandoCobrarComanda	Orquestar el cobro completo de una comanda.	RepositorioDeComandas, CalculadoraDeTotal, SelectorDePoliticaDePago, RepositorioDePedidos, RepositorioDeMesas
Comanda (del flujo 2)	Decidir si puede cobrarse y cerrarse.	DetalleDeComanda
CalculadoraDeTotal	Obtener el total sumando los subtotales de una comanda.	Ninguno
SelectorDePoliticaDePago	Elegir la política que corresponde al tipo de pago recibido.	PoliticaDePagoEnEfectivo, PoliticaDePagoConTarjeta
PoliticaDePagoEnEfectivo	Decidir si el efectivo alcanza y calcular el cambio.	Ninguno
PoliticaDePagoConTarjeta	Aceptar el pago exacto, sin cambio.	Ninguno
Pedido	Representar un cobro ya realizado (pago, tipo, cambio, fecha).	Ninguno
RepositorioDePedidos	Guardar los cobros realizados.	Pedido
ManejadorDeConsultaComandasPorCobrar	Armar la lista de comandas con su total para caja.	LectorDeComandasPorCobrar
LectorDeComandasPorCobrar	Entregar las comandas por cobrar con su total ya calculado.	Ninguno
ManejadorDeConsultaTicket	Armar los datos del ticket de un cobro ya hecho.	LectorDeTickets
LectorDeTickets	Entregar los datos de un cobro en forma de ticket.	Ninguno
ImpresorDeTicket	Dibujar el ticket y mandarlo a imprimir.	Ninguno

Referencia de origen: flujos-cqrs-crc_original.pdf, páginas 5 y 6.

OBSERVACIONES DE CONSISTENCIA

El contrato OpenAPI 3.0.3 documenta 16 operaciones y no expone un endpoint separado para consultar permisos. Los permisos forman parte de LoginResponse. Los roles y el acceso a cocina requieren una decisión adicional. El esquema de cierre de caja aún recibe totalSistema desde el cliente y conviene revisarlo antes de programar.

Por ello, este material es un diseño de referencia. La verificación de peticiones HTTP reales exige implementar controllers, servicios, repositorios, base de datos e infraestructura de autenticación.